Pure Insights Command Centre

An admin dashboard that restores the visibility lost in the Tableau migration: report health, orphaned-report ownership, real-time alerting, and Snowflake cost attribution per report. Submitted by **Abhyuday Purohit**.

Code & live repository
github.com/Abhyuday007/everpure-technical-data-ai-validation-pure-insights-command-centre

3 / 3	15 / 15	2,000	5 / 5
required deliverables: code, API spec, SQL suite	backend tests passing (alert rules, RBAC, WebSocket)	rows validated; 10 issue types logged and handled	cost reconciliation checks passing

What I found first, and why it shaped the design

- **The file is 100 reports × 20 snapshots, not 2,000 reports.** Values change between repeats, so I kept them as snapshots instead of de-duplicating them away.
- **Ownership is missing in two encodings** (`nan` and `ORPHANED_REPORT`): 245 snapshots. Checking only one would undercount.
- **74.5% of rows exceed the 15s latency threshold.** A flat alert rule would fire on three events in four, so alerts use warning and critical tiers plus a per-report cooldown.
- **All numeric columns are uncorrelated**, so the data is synthetic. The analytics rank and flag; they make no causal claims.

Run it in two commands

```
npm run setup   # install root, backend, frontend
npm run dev     # API on :4000, web app on :5173
```

ROLE	EMAIL	PASSWORD	CAN DO
admin	`admin@pureinsights.local`	`Admin#123`	Everything, incl. assign owners, control simulator
analyst	`analyst@pureinsights.local`	`Analyst#123`	Reports, orphans, costs, data quality, acknowledge alerts
viewer	`viewer@pureinsights.local`	`Viewer#123`	Overview and live alerts only

To see live alerts: sign in as admin → Live alerts → Start. Demo accounts are hard-coded for the assessment (stated in the limitations).

REQUIREMENTS TRACEABILITY

Every requirement in the brief, and where to verify it

Status is stated plainly, including where something is simulated or where I deviated from the brief.

BRIEF REQUIREMENT	WHAT WAS BUILT	WHERE	STATUS
Task 1. React + Node/Express dashboard with RBAC to monitor system health and orphaned reports	Six screens (overview, live alerts, reports, orphaned reports, cost allocation, data quality). JWT login; three roles enforced **on the server**, with the UI only mirroring it. Admins can assign an owner to an orphaned report.	`frontend/, backend/src/app.js, auth.js`	MET
Task 2. WebSocket/SSE alerts when load > 15s or a refresh fails	Authenticated WebSocket pushes alerts live; acknowledgements broadcast to all clients. Latency >15s = warning, >30s = critical; refresh FAILED = critical, TIMED_OUT = warning.	`backend/src/alertEngine.js, simulator.js`	MET*
Task 3. SQL on simulated `ACCOUNT_USAGE` to attribute credits to reports and compute unit cost per active-user view	Execution-time-share allocation per warehouse and period; cost per active user view with explicit zero-view handling; reconciliation suite; Snowflake production version of the same logic.	`sql/01–06, snowflake_production.sql`	MET*
Clean injected defects: orphans, negative credits, latency spikes, user-vs-AWS-cost discrepancy	245 orphaned snapshots, 24 negative credit values, 910 snapshots over 30s (max 55s), 100 high AWS-cost-per-view snapshots flagged. Nothing dropped silently.	`notebooks/01_data_validation.ipynb, validation/cleaning.py`	MET

Required submission deliverables

DELIVERABLE	LOCATION	STATUS
Runnable full-stack codebase (React frontend, Node backend)	`frontend/ · backend/ · README quick start`	DELIVERED
REST API and WebSocket technical specification	`docs/API_SPEC.md`	DELIVERED
Snowflake cloud cost allocation SQL script suite	`sql/ (01–06 + snowflake_production.sql)`	DELIVERED

* The two asterisks, stated up front

- **Alerting is a replay.** The dataset has no timestamps, so a simulator feeds rows through the same code path a live feed would use. Nothing is monitored for real.
- **Cooldown vs the brief.** The brief says to alert when load exceeds 15s. I add a 90-second per-report cooldown (configurable) that suppresses repeats unless severity escalates, because a flat rule fires on ~75% of rows. The 30s critical tier and the cooldown are my design choices, not from the brief.
- **Snowflake is simulated.** No Snowflake account was available. The suite runs on DuckDB against tables that use Snowflake's real column names. The production SQL parses as Snowflake SQL but has not been executed.

DATA VALIDATION

Ten issue types found, logged and handled

Rule: never silently drop a row. A bad value is flagged, and excluded only from the one calculation it would corrupt.

ISSUE	ROWS	% ROWS	HANDLING
Orphaned report, owner `nan`	128	6.4	Flag `is_orphaned`, keep row
Orphaned report, `ORPHANED_REPORT`	117	5.9	Same flag: two encodings of one condition
Negative Snowflake credits	24	1.2	Keep row, null the value, exclude from cost sums
Refresh status `UNKNOWN`	100	5.0	Own status; never counted as success or failure
Load time 15–30s	580	29.0	Warning-tier alert, with cooldown
Load time > 30s (max 55s)	910	45.5	Critical-tier alert
Zero monthly views	4	0.2	Per-view metrics set to NULL (no divide-by-zero)
Peak concurrent users > monthly views	93	4.6	Flag as suspicious; keep values
AWS cost per view above 95th percentile (> $2.64)	100	5.0	Flag as cost-vs-usage mismatch for review
Report IDs repeated across rows	1,900	95.0	Not deleted: treated as snapshots (`snapshot_seq`)

Source: `data/dq_issues.csv`, produced by the validation notebook. I re-derived the headline counts (245, 24, 100, 55s, 74.5%) directly from the raw CSV.

Decisions I made, and what would change them

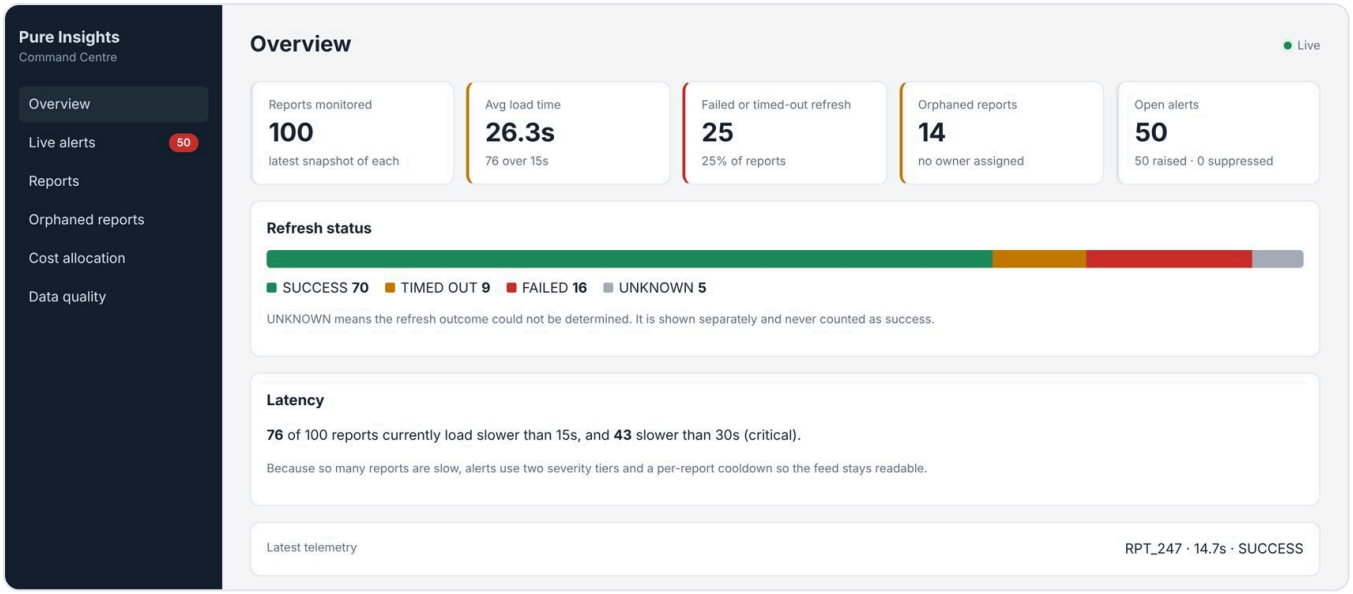
- **Snapshot order is file order.** With no timestamp, the last row per report is its "current" state. A report is *currently orphaned* if that last snapshot has no owner. No report is orphaned in all 20 snapshots, so this rule matters: a different one gives a different list.
- **Negative credits are errors, not refunds.** If they are legitimate refunds they should be netted instead; the assumption is logged with its effect.
- **UNKNOWN is not success.** Counting it as failure would add 5% more alerts.

Checks beyond the brief

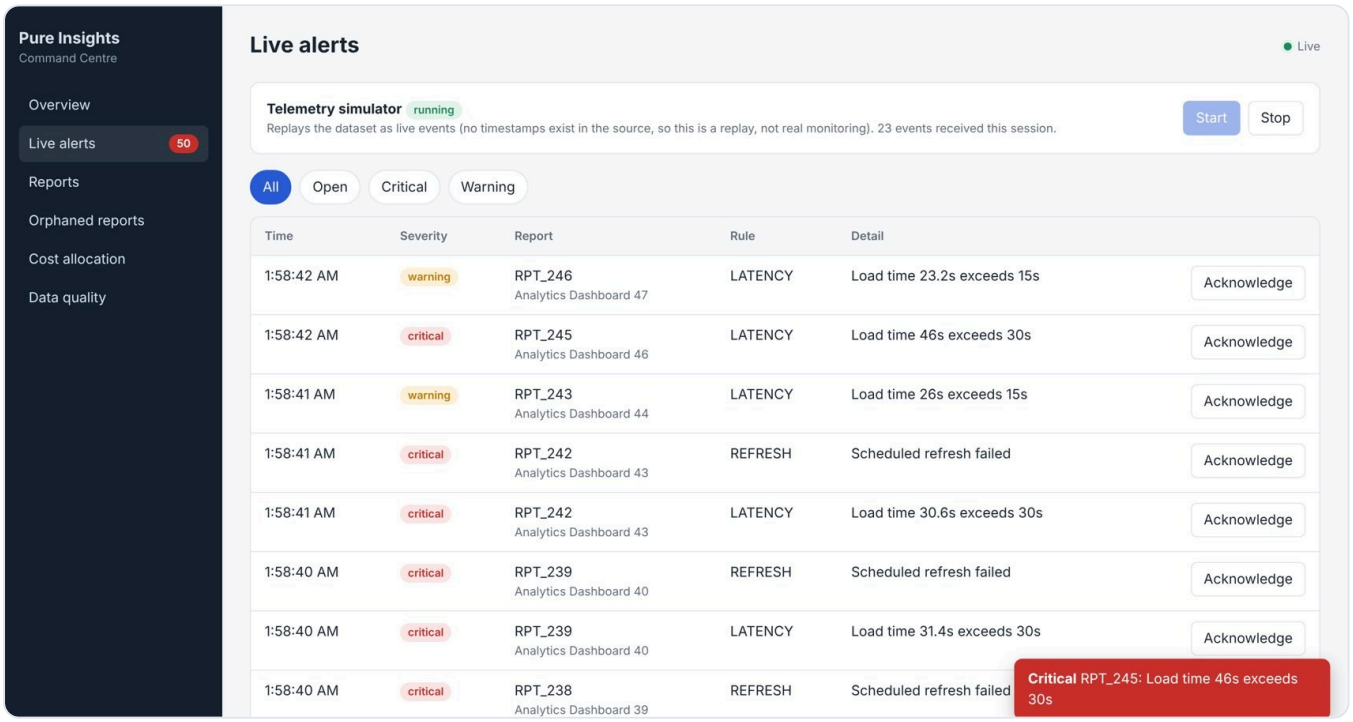
- Zero-view rows and the divide-by-zero risk they create in unit cost.
- Concurrent peak above total monthly views, which is not plausible.
- Pairwise correlation across all numeric columns, which showed the data is synthetic and shaped what the dashboard is allowed to claim.
- The same cleaning rules live in one shared module (`validation/cleaning.py`), used by the notebook and the pipeline, so there is one source of truth.

THE APPLICATION

Overview and live alerting



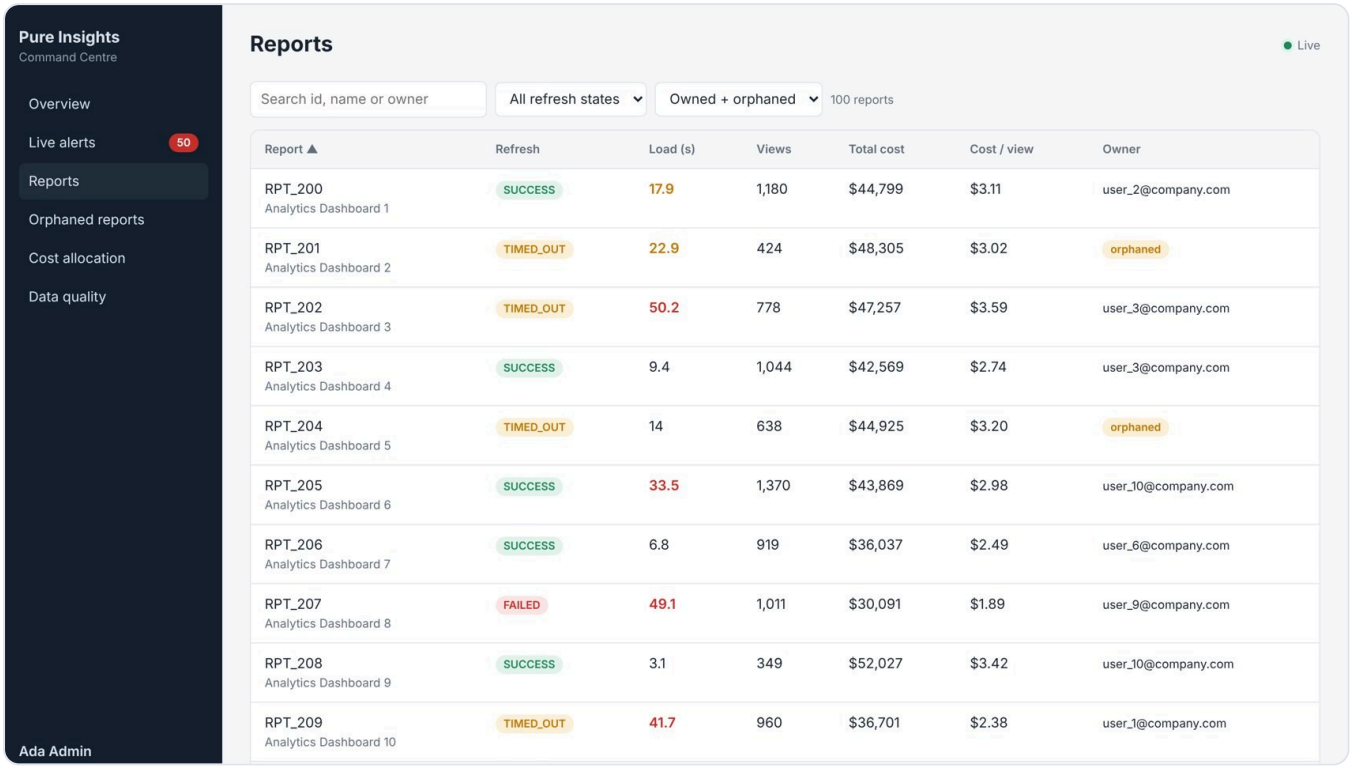
Overview. Health across 100 reports at their latest snapshot: 76 load slower than 15s and 43 slower than 30s. UNKNOWN refresh outcomes are shown as their own state, never as success.



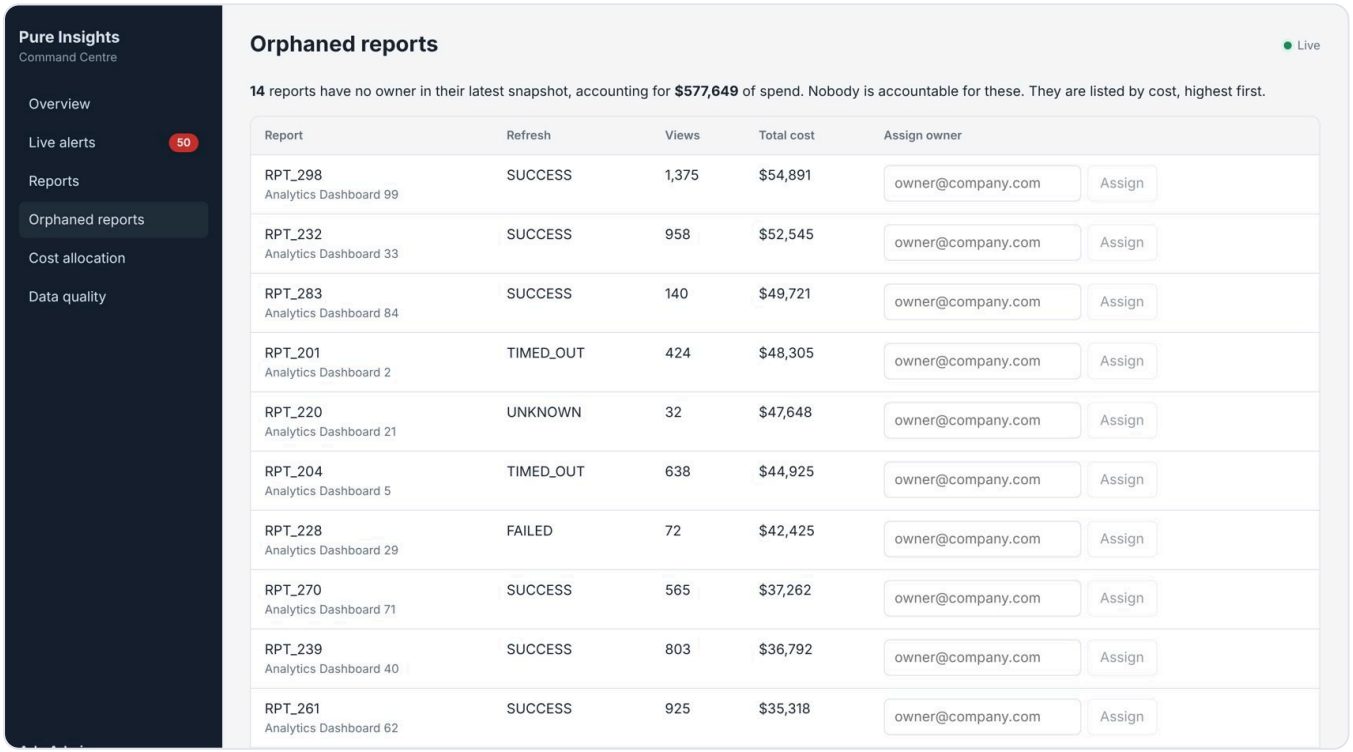
Live alerts. WebSocket feed with critical toasts and severity filters. Each alert carries rule, value and threshold, and analysts can acknowledge it; the acknowledgement is broadcast to every connected client. The banner states that this is a replay of the dataset, not real monitoring.

THE APPLICATION

Reports and orphaned-report ownership



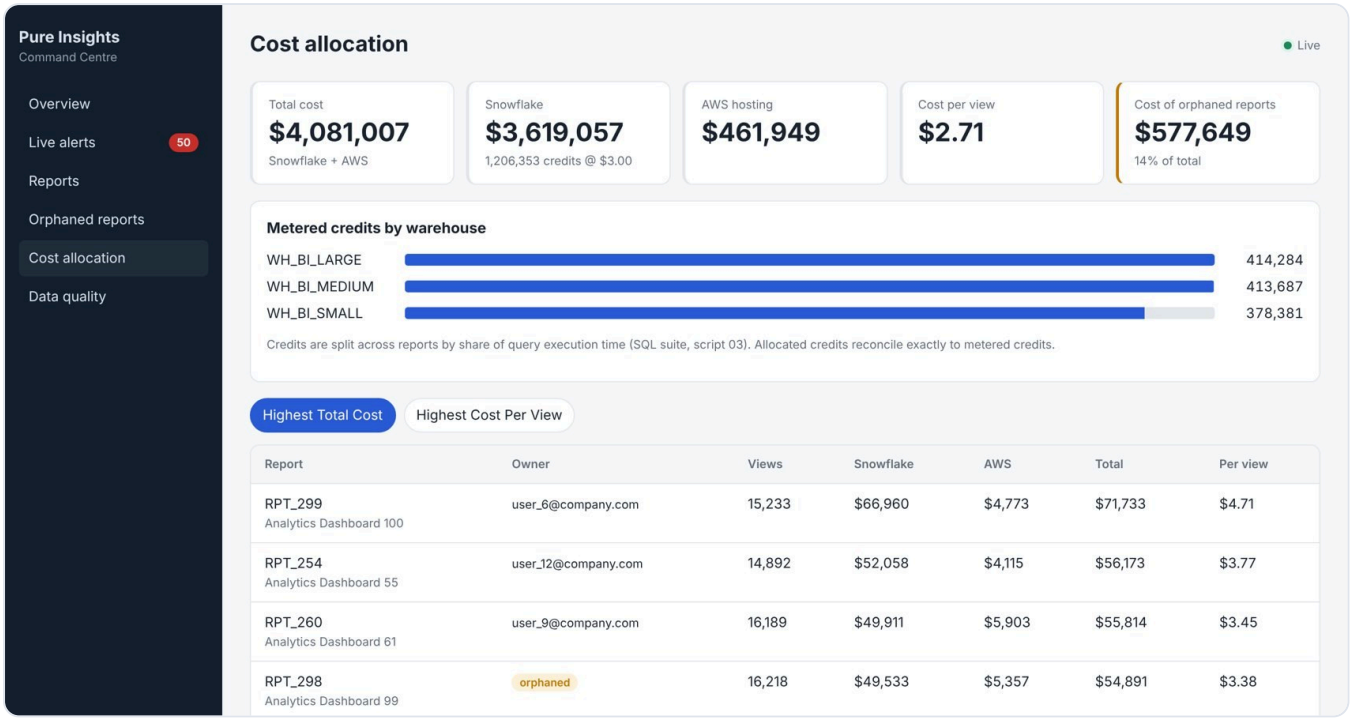
Reports. Search, filter by refresh state and ownership, sort by load time, cost and cost per view, with server-side pagination. Load times are coloured by severity tier.



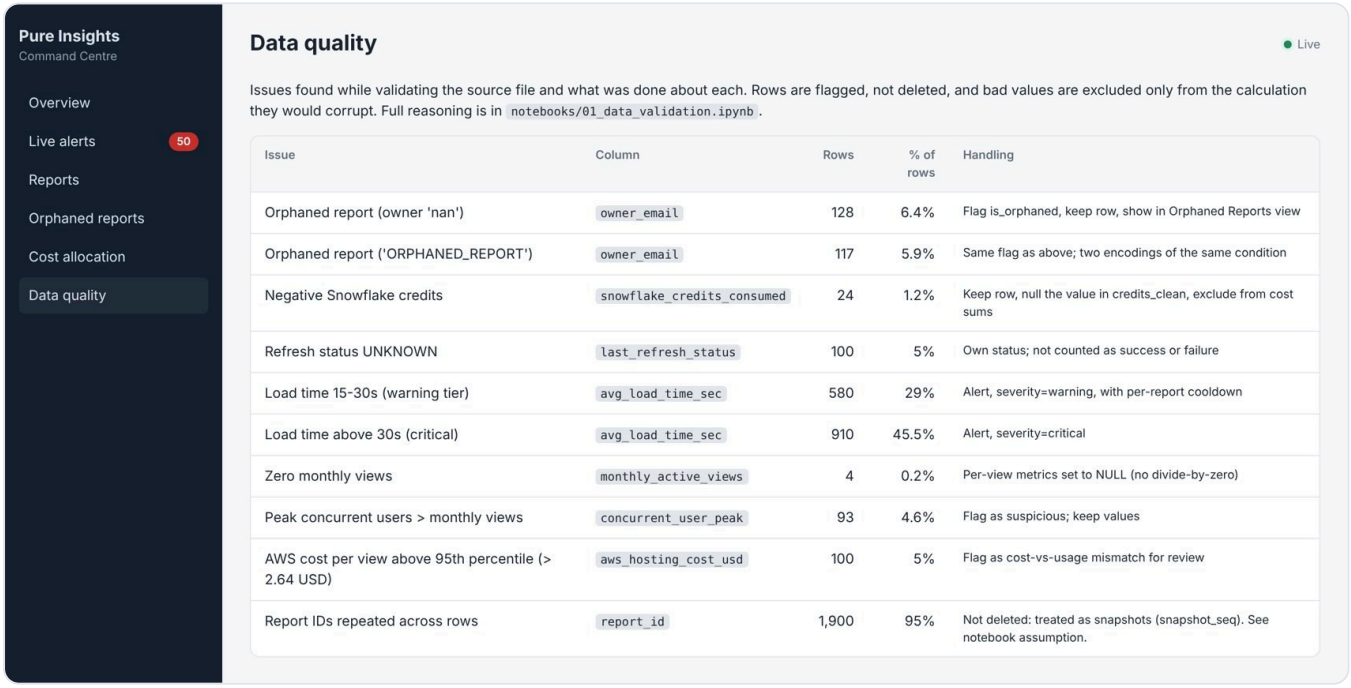
Orphaned reports. 14 reports have no owner in their latest snapshot and account for $577,649 of spend (14% of total), listed highest cost first. Admins assign an owner inline; the server validates the address and enforces the admin role.

THE APPLICATION

Cost allocation and data quality



Cost allocation. Total, Snowflake, AWS, cost per view, and orphaned spend, with metered credits per warehouse. Dollar figures use a $3.00 per credit placeholder and simulated warehouse assignment (see page 8); treat them as a method demonstration.

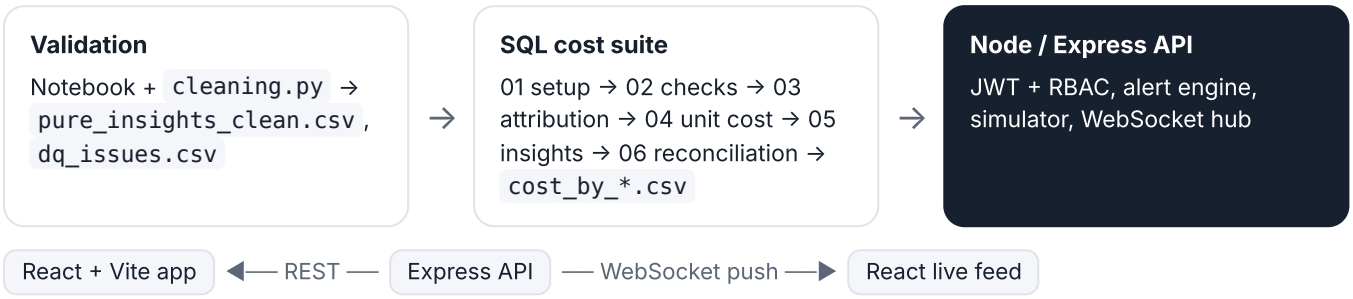


Data quality. The validation log is surfaced inside the product, not buried in a notebook, so an analyst can see what was flagged, how many rows, and what was done.

SYSTEM DESIGN

Architecture, access control and API

Cleaning logic and cost logic each live in exactly one place. The backend serves what they produce and does not re-implement either.



Role-based access (enforced in middleware)

CAPABILITY	VIEWER	ANALYST	ADMIN
Health summary, alerts, live feed	✓	✓	✓
Reports, orphans, costs, data quality		✓	✓
Acknowledge alerts		✓	✓
Assign owner, control simulator			✓

The RBAC matrix has its own automated test, so a loosened rule fails the build.

Alert rules (pure, unit-tested logic)

RULE	TRIGGER	SEVERITY
LATENCY	> 15s / > 30s	warning / critical
REFRESH	TIMED_OUT / FAILED	warning / critical
REFRESH	UNKNOWN, SUCCESS	no alert

Repeats for the same report and rule are suppressed for 90s unless severity escalates. Thresholds and cooldown are environment variables.

API surface (full spec in `docs/API_SPEC.md`)

ENDPOINT	ROLE	PURPOSE
`POST /api/auth/login`	public	Returns a JWT (8h default)
`GET /api/health/summary`	any	Current state across reports; alert and simulator stats
`GET /api/reports · /orphaned · /:id`	analyst+	Filter, sort, paginate (validated, allow-listed sort keys); per-report history
`PATCH /api/reports/:id/owner`	admin	Assign owner (email validated); broadcasts `owner_assigned`
`GET /api/costs/summary · /reports`	analyst+	Totals, per-warehouse credits, top reports by cost or cost per view
`GET /api/alerts · POST /api/alerts/:id/ack`	any / analyst+	List and acknowledge alerts
`WS /ws?token=…`	any	Server push: `telemetry`, `alert`, `alert_ack`, `owner_assigned`, `simulator_status`. Bad token refused with HTTP 401 at upgrade.

Errors share one shape, `{error, message}`, with 400/401/403/404/500 documented. Request bodies are capped at 10 KB.

SNOWFLAKE COST ATTRIBUTION

Attribute warehouse credits to reports, then prove nothing leaks

Snowflake bills credits per warehouse, not per dashboard. Report-level cost is therefore an **attribution model**, not a metered figure.

```
report_credits = warehouse_metered_credits × report_execution_ms ÷
total_execution_ms_on_warehouse   (per period)
cost_usd = report_credits × credit_price
cost per active user view = cost_usd ÷ monthly_active_views   (NULL when views = 0)
```

Why execution time. One 40-second dashboard load keeps a warehouse running far longer than forty 1-second lookups, and compute time is what Snowflake bills. I deliberately did not build a weighted formula of query count × latency × concurrency × AWS cost: it would look sophisticated and be harder to defend. "Active user view" is `monthly_active_views`, the only view field supplied.

Reconciliation output (`06_reconciliation.sql` , run on the repo)

WAREHOUSE	METERED CREDITS	ALLOCATED CREDITS	DIFFERENCE	STATUS
WH_BI_LARGE	414,284.43	414,284.43	0.00	PASS
WH_BI_MEDIUM	413,687.35	413,687.35	0.00	PASS
WH_BI_SMALL	378,380.70	378,380.70	0.00	PASS
Total	**1,206,352.48**	**1,206,352.48**	**0.00**	PASS

Five checks pass in total: overall conservation, every warehouse-and-period cell, elapsed shares summing to 1, no negative metering, and the cost table tying back to metered credits × price. **I also corrupted one report's allocation on purpose and confirmed the checks went red** (20 leaking cells, +134.9 credits), so the control can actually fail.

What this proves, and what it does not

Proves: the allocation neither creates nor loses credits.

Does not prove: that the metering data is accurate, or that execution time is the true cost driver. In this repo it is also weaker than on a real account, because simulated metering is built from the same credits.

Simulation vs production

Report → QUERY_TAG → QUERY_HISTORY

warehouse + hour → METERING_HISTORY → allocated credits

Runs locally: `sql/01–06` on DuckDB. Production mapping: `snowflake_production.sql` , which assumes the portal sets `QUERY_TAG = 'report_id=<ID>'` , keeps untagged and idle-hour credits visible, and notes that `ACCOUNT_USAGE` lags real time. Syntax-checked with a Snowflake parser; not executed.

Resulting figures (method demonstration)

Total cost **$4,081,007** = Snowflake **$3,619,057** (1,206,353 credits at a $3.00 placeholder) + AWS **$461,949**; **$2.71** per view overall. Highest cost: RPT_299 at $71,733 ($4.71 per view). The warehouse assignment (small, medium, large by report number) is invented, so per-report attribution is illustrative.

HOW I VALIDATE

Checking the output, not just producing it

The role is about trusting analytics only after they are independently verified. These are the habits this project demonstrates.

PRACTICE	HOW IT SHOWS UP IN THE PROJECT
Recompute independently	The notebook (pandas) and the SQL suite (DuckDB) both derive the data-quality counts; `02_clean_filter.sql` must reproduce `dq_issues.csv`. A backend test checks that the cost summary served by the API equals the SQL suite's output.
Flag, don't delete	Orphans, negative credits and suspicious rows are flagged and traceable. A value is excluded only from the calculation it would corrupt.
Make assumptions falsifiable	`ASSUMPTIONS.md` lists each assumption with the effect if it is wrong, so a reviewer can disagree quickly.
Controls that can fail	Reconciliation is a pass/fail suite, and I verified it by breaking the allocation on purpose.
Don't over-claim	The data is shown to be synthetic, so analytics rank and flag instead of asserting causes. Simulated and untested pieces are labelled as such throughout.
Human in the loop	Ambiguous states are surfaced for a person, not resolved silently: UNKNOWN refresh status, orphaned reports awaiting an owner, and alerts that need acknowledgement.

Limitations, stated up front

- No Snowflake account: SQL ran on DuckDB, and the production SQL is unexecuted.
- "Real-time" is a replay of dataset rows.
- Owner assignments and alerts live in server memory and reset on restart; single process only.
- Hard-coded demo users; no rate limiting, audit log or token revocation. JWT secret falls back to a dev default with a warning.
- The WebSocket token travels in the query string (browsers cannot set headers); use `wss://` in any real deployment.
- No browser-level UI tests; the UI was exercised manually.
- Credit price ($3.00) and warehouse mapping are placeholders.

Next steps if this went to production

Proposed, not built.

- Persist owners and alerts in a table, and add SSO instead of demo users.
- Tag queries with the report ID at the query layer and run the production SQL on real `ACCOUNT_USAGE`.
- Feed alerts from real telemetry instead of the replay.
- Add automated recompute checks that diff application outputs against Snowflake results, with a review gate that holds questionable figures before they reach decision-makers.
- Add browser tests and CI.

Repository map. `frontend/` React app · `backend/` API, auth, alert engine, WebSocket, tests · `sql/` cost suite · `notebooks/` validation · `validation/` shared cleaning rules · `docs/` API spec, architecture, assumptions · `data/` raw, clean and derived files.
github.com/Abhyuday007/everpure-technical-data-ai-validation-pure-insights-command-centre